Java Programming
CO2-AT1

Name: Preethika M
Reg No: 192421084

A food-delivery application maintains a dynamic list of active orders. New orders are appended frequently, while cancelled orders may be removed from the middle. Evaluate whether ArrayList is appropriate and discuss the performance trade-offs.
[ArrayList Performance]

ArrayList Performance in a Food-Delivery Application

A food-delivery application maintains a dynamic list of active orders. New orders are continuously added to the system, while cancelled orders may need to be removed from any position in the list. An **ArrayList** can be used for this purpose because it provides dynamic resizing and efficient access to elements. However, its suitability depends on how frequently insertion, deletion, and access operations are performed.

1. Adding New Orders

New orders are usually appended to the end of the ArrayList. Adding an element at the end is generally very efficient and takes **O(1) amortized time**.

However, when the internal array becomes full, the ArrayList must create a larger array and copy the existing elements into it. This resizing operation takes **O(n)** time. Since resizing happens only occasionally, adding elements is still considered **O(1) amortized**.

Therefore, ArrayList performs well when new food orders are frequently added to the end of the active-order list.

2. Removing Cancelled Orders

The main disadvantage of using an ArrayList occurs when a cancelled order needs to be removed from the middle of the list.

Suppose the active orders are:

[Order1, Order2, Order3, Order4, Order5]

If **Order3** is cancelled, it is removed:

[Order1, Order2, Order4, Order5]

After removing Order3, Order4 and Order5 must be shifted one position to the left to fill the empty space. Therefore, removing an element from the middle requires shifting the remaining elements.

The time complexity for this operation is **O(n)**.

If cancellations happen frequently, these repeated shifting operations can reduce the performance of the application, especially when the number of active orders is very large.

3. Accessing Orders

ArrayList provides very efficient random access. Any order can be accessed directly using its index.

For example:

```
orders.get(3)
```

The time complexity for accessing an element by index is **O(1)**.

This is one of the major advantages of ArrayList compared with data structures such as LinkedList.

4. Searching for an Order

If the application searches for a particular order by its order ID using a normal linear search, it may have to check several elements before finding the required order.

Therefore, searching an ArrayList generally takes **O(n)** time.

For applications with thousands of active orders, another data structure such as a **HashMap** may be more efficient when frequent lookup by order ID is required.

Performance Trade-offs

Operation	ArrayList Time Complexity	Performance
Add order at end	O(1) amortized	Very efficient
Access order by index	O(1)	Very efficient
Search for an order	O(n)	Moderate for large lists
Remove last order	O(1)	Efficient
Remove order from middle	O(n)	Less efficient
Resize internal array	O(n)	Occurs occasionally

Advantages of Using ArrayList

- It automatically increases its size when new orders are added.
- Adding new orders at the end is very fast.
- Orders can be accessed directly using their index.
- It is simple to implement and maintain.
- It generally provides good performance when additions are more frequent than deletions.

Disadvantages of Using ArrayList

- Removing cancelled orders from the middle takes **O(n)** time.
- Elements must be shifted after a middle deletion.
- Frequent resizing may temporarily increase processing time.
- Searching for an order by ID takes **O(n)** without an additional indexing mechanism.
- Frequent middle deletions can become inefficient when the list contains a large number of active orders.

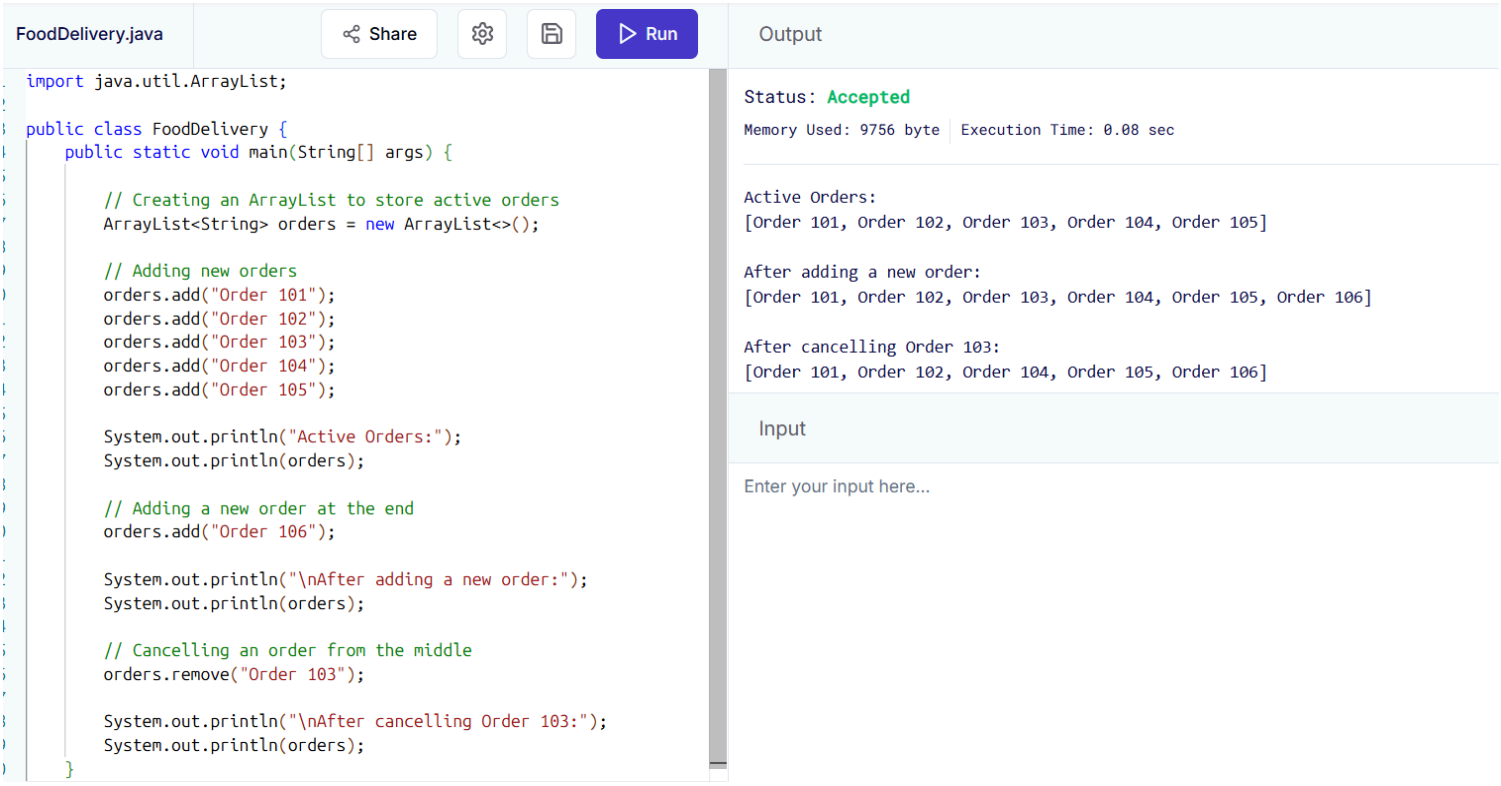
Conclusion

An **ArrayList is reasonably appropriate** for a food-delivery application when new orders are frequently appended and cancellations are relatively less frequent. Its **O(1) amortized insertion at the end** and **O(1) random access** make it efficient for adding and accessing active orders.

However, if cancelled orders are frequently removed from the middle, ArrayList becomes less efficient because each deletion may require shifting many elements, resulting in **O(n) time complexity**.

Therefore, ArrayList is a good choice when the application performs **more additions and accesses than middle deletions**. If frequent deletion or fast lookup by order ID is a major

requirement, alternative data structures such as **LinkedList or HashMap** should also be considered.



Code:

```
import java.util.ArrayList;

public class FoodDelivery {
    public static void main(String[] args) {

        // Creating an ArrayList to store active orders
        ArrayList<String> orders = new ArrayList<>();

        // Adding new orders
        orders.add("Order 101");
        orders.add("Order 102");
        orders.add("Order 103");
        orders.add("Order 104");
        orders.add("Order 105");

        System.out.println("Active Orders:");
```

```
        System.out.println(orders);

        // Adding a new order at the end
        orders.add("Order 106");

        System.out.println("\nAfter adding a new order:");
        System.out.println(orders);

        // Cancelling an order from the middle
        orders.remove("Order 103");

        System.out.println("\nAfter cancelling Order 103:");
        System.out.println(orders);
    }
}
```